

LEGAL EASE

AI-Powered Legal Document Generator

PROJECT DOCUMENTATION

A project report covering objectives, use cases, architecture, development milestones, implementation, testing, and future enhancements.

1. Introduction

Legal Ease is an AI-powered application designed to simplify the creation of legal documents using Generative AI. Users provide basic information such as document type, parties, terms and conditions, and important dates. The application uses Google Gemini to generate structured document content.

Users can review and edit the generated content and download it in formats such as PDF, DOCX, and TXT. The system aims to make document preparation more accessible to people without a legal background.

2. Project Objectives

- Develop an AI-powered legal document generator.
- Automate the creation of common legal documents.
- Integrate Generative AI through the Gemini API.
- Build a user-friendly interface with Streamlit and a backend with FastAPI.
- Support document preview, editing, formatting, and downloads.
- Reduce the time and effort involved in preparing basic legal documents.

3. Project Overview

Legal Ease accepts user-provided information and generates a structured legal document tailored to the selected document type. Its main features include AI-powered generation, multiple document types, user input collection, preview and editing, professional formatting, and PDF, DOCX, and TXT downloads.

4. Real-World Use Cases

Scenario 1: Startup Founder

A startup founder enters an employee's details, job role, compensation, and confidentiality terms. The application generates an employment contract that can be reviewed, customized, and downloaded.

Scenario 2: Freelancer

A freelancer enters the parties' names, confidentiality requirements, and agreement details. Legal Ease generates a structured Non-Disclosure Agreement (NDA) for review and editing.

Scenario 3: Landlord

A landlord enters the property address, tenant details, rental amount, and lease duration. The application generates a formatted residential lease agreement that can be edited and downloaded.

Important: AI-generated legal documents should be reviewed by a qualified legal professional before signing or use.

5. Prerequisites

Technology / Tool	Purpose
Python	Main programming language
Streamlit	Frontend user interface
FastAPI	Backend API development
Google Gemini API	AI document generation
Pydantic	Request data validation
python-docx	DOCX document generation

Technology / Tool	Purpose
ReportLab	PDF generation
VS Code	Code editor
Git and GitHub	Version control and project hosting

6. Project Workflow

Activity	Description
1. Model Selection and Architecture	Select an AI model and design the application architecture.
2. Core Functionality Development	Develop generation, editing, formatting, and downloading features.
3. Backend and API Integration	Connect FastAPI with the Gemini model.
4. Frontend Development	Build the user interface using Streamlit.
5. Deployment and Testing	Run the application locally and test its functionality.
6. Conclusion	Evaluate the completed application and identify future enhancements.

7. System Architecture

The application is organized into four major components:

Component	Responsibility
Frontend – Streamlit	Collects user inputs and provides document preview, editing, and downloads.
Backend – FastAPI	Receives frontend requests, validates data, and coordinates processing.
AI Core – Gemini	Generates legal document content from user inputs.
Document Formatting Module	Converts generated content into PDF, DOCX, or TXT files.

Typical flow: User → Streamlit frontend → FastAPI backend → Gemini AI → document formatting → preview/download.

8. Milestone 1: Model Selection and Architecture

8.1 AI Model Selection

The project documentation proposes Google Gemini 1.5 Pro for structured legal document generation, citing its long-context processing and text-generation capabilities. For a new implementation, verify currently available Gemini models and API compatibility before choosing a model.

8.2 Application Architecture

A modular architecture separates the frontend, backend, AI generation, and document formatting components. This separation supports maintainability, testing, and future extension.

8.3 Environment Setup

Install Python and VS Code, create a virtual environment, install the required packages, and configure the Gemini API key securely. Do not commit API credentials to a public repository.

9. Milestone 2: Core Functionality Development

9.1 Legal Document Generation

Users provide the document type, parties, terms and conditions, important dates, and any additional requirements. The backend passes the information to Gemini to generate the document.

9.2 Document Formatting

Generated content is converted into a professional layout with appropriate headings, paragraphs, clauses, and sections.

9.3 User Interaction

The Streamlit interface collects user preferences and requirements and displays the generated result.

9.4 Editing and Downloading

Users can review and modify generated content before downloading it in their preferred format.

10. Milestone 3: Backend and API Integration

10.1 Main Backend Components

Component	Responsibility
main.py	Initializes the FastAPI application.
routes.py	Defines API endpoints.
Request model	Validates incoming user data using Pydantic.
Gemini generator	Generates legal document content.
Document utilities	Formats and exports documents.

10.2 API Request Processing

1. Receive the document generation request.
2. Validate the input using Pydantic.
3. Pass validated information to the Gemini generator.
4. Generate legal document content.
5. Return the content to the frontend.
6. Handle validation, API, and generation errors.

10.3 API Endpoint

Example endpoint: **POST /generate**. It accepts document details and returns generated content.

11. Milestone 4: Frontend Development

11.1 Page Layout

The Streamlit page includes the application logo and title, document type selection, party details, terms and conditions, date selection, a Generate Document button, document preview, editing controls, and download buttons.

11.2 Example User Input

Field	Example
Document Type	Freelance Work Contract
Service Provider	Jane Doe

Field	Example
Client	Technova
Payment Terms	Payment within 30 days
Confidentiality	Must be maintained
Delivery	Work should be delivered on time
Date	April 10, 2026

11.3 Preview, Editing, and Downloads

After generation, the document is displayed for review. Users can edit the content and download it in PDF, DOCX, or TXT format.

12. Milestone 5: Deployment and Testing

12.1 Create a Virtual Environment

Create an isolated Python environment:

```
python -m venv venv
```

On Windows, activate it with:

```
venv\Scripts\activate
```

12.2 Install Dependencies

Install the packages listed in the project requirements file:

```
pip install -r requirements.txt
```

12.3 Configure the Gemini API Key

Create a .env file and store the API key:

```
GEMINI_API_KEY=your_api_key
```

Keep .env out of source control and never expose the key in screenshots, logs, or a public repository.

12.4 Run the Backend

```
uvicorn backend.main:app --reload
```

12.5 Run the Frontend

```
streamlit run frontend/app.py
```

Adjust these commands if the final project uses a different package or folder structure.

12.6 Application Testing

Test Case	Expected Result
Valid document details	Inputs are accepted.
Generate document	Generated content is displayed.
Incomplete details	A validation message is shown.
Edit content	Updated content is displayed.
Download PDF / DOCX / TXT	The requested file is generated.
Invalid API key	An appropriate error is shown.

Test Case	Expected Result
API unavailable	The application handles the failure gracefully.

13. Project Folder Structure

```
Legal-Ease/  
├── backend/  
│   ├── main.py  
│   ├── routes.py  
│   └── ai_core/  
│       ├── gemini_generator.py  
│       ├── document_utils/  
│       │   ├── pdf_generator.py  
│       │   ├── docx_generator.py  
│       │   └── text_generator.py  
│       └── frontend/  
│           ├── app.py  
│           ├── assets/  
│           └── logo.png  
├── .env  
├── .gitignore  
├── requirements.txt  
└── README.md
```

This is a proposed structure based on the project description. The actual implementation may contain additional files.

File / Folder	Description
backend/	Backend application files.
main.py	Starts the FastAPI server.
routes.py	Handles API requests.
ai_core/	AI-related functionality.
gemini_generator.py	Generates legal content using Gemini.
document_utils/	Document formatting and export functions.
frontend/	Streamlit application files.
app.py	Main user interface.
assets/	Images and logos.
.env	Stores API credentials.
.gitignore	Excludes sensitive and unnecessary files.
requirements.txt	Lists Python dependencies.
README.md	Project setup and usage instructions.

14. Project Management Using Skill Wallet

Skill Wallet is used to organize and track project development. Its Overview section provides a high-level summary of project activities and status. Workspace is the main area for managing tasks, while the Kanban Board visualizes task progress.

Kanban Column	Purpose
In Progress	Tasks currently being developed.
To Be Reviewed	Completed tasks submitted for review.

Kanban Column	Purpose
Completed	Tasks validated and approved.

As tasks are completed, move them through the appropriate columns. Submit the GitHub repository, documentation, and demo video links for review. The project completion status is validated after submission.

15. Future Enhancements

- Advanced Contract Analysis: Analyze documents and identify important clauses.
- Legal Database Integration: Include relevant legal references.
- Personalized Recommendations: Suggest document options based on user requirements.
- Multiple Language Support: Generate documents in different languages.
- User Authentication: Securely save and access user documents.
- Cloud Deployment: Make the application accessible online.

16. Conclusion

Legal Ease combines Python, Streamlit, FastAPI, and Google Gemini AI to simplify legal document preparation. It collects user information and generates structured content, while supporting document preview, editing, formatting, and downloading.

The project provides practical experience in Generative AI integration, API development, frontend development, document processing, and software testing. Although it can simplify document preparation, AI-generated legal documents should be reviewed by a qualified legal professional before use or signing.

Project Summary

Category	Technology
Programming Language	Python
Frontend	Streamlit
Backend	FastAPI
AI Model	Google Gemini
Validation	Pydantic
Output Formats	PDF, DOCX, TXT
IDE	VS Code
Version Control	GitHub